



API REST de Productos



UNIVERSIDAD POLITÉCNICA DE PACHUCA

Estándares y Métricas para el Desarrollo de Software

Documentación técnica y plan de pruebas—Productos con imagen polimórfica
uno a uno (Laravel 12)

proyecto	store-api — API REST de Productos
Alumno	Eduardo Hernández Gutiérrez
Grupo	TIID_01_05
Docente	Miguel Ángel Montoya Cerro
Fecha	10/julio/226
Puntaje Total	

Contenido

1. Objetivo
2. Arquitectura y ambiente
3. Estructura de datos
4. Modelos y relación polimórfica
5. Endpoints de la API
6. Validaciones
7. Evidencia visual: galería de productos
8. Plan de pruebas (Plantilla 12.1)
9. Casos de prueba (Plantilla 12.2)
10. Matriz de seguimiento (Plantilla 12.3)
11. Reporte de defectos (Plantilla 12.4)
12. Tipos de prueba propuestos
13. Conclusión técnica

Objetivo General

Desarrollar una API REST para la gestión de productos de un comercio electrónico utilizando el framework Laravel 12 y PostgreSQL, implementando operaciones CRUD, almacenamiento de imágenes mediante relaciones polimórficas y validación de datos, aplicando buenas prácticas de desarrollo backend.

Objetivos Específicos

Configurar un proyecto Laravel conectado a una base de datos PostgreSQL.

Diseñar e implementar la estructura de la base de datos mediante migraciones.

Desarrollar los endpoints necesarios para realizar las operaciones de Crear, Consultar, Actualizar y Eliminar productos.

Implementar validaciones de datos utilizando Form Requests.

Gestionar la carga y almacenamiento de imágenes mediante el sistema Storage de Laravel.

Aplicar relaciones polimórficas para asociar imágenes a los productos.

Verificar el funcionamiento de la API mediante pruebas realizadas con Postman.

Tecnologías utilizadas

- Tecnología Descripción
- Laravel 12 Framework PHP utilizado para el desarrollo de la API REST.
- PHP 8.2 Lenguaje de programación empleado para el backend.
- PostgreSQL Sistema gestor de base de datos relacional.
- Postman Herramienta utilizada para probar los endpoints de la API.
- Composer Administrador de dependencias de PHP.
- pgAdmin 4 Herramienta gráfica para administrar PostgreSQL.
- Storage de Laravel Sistema para el almacenamiento y gestión de archivos.

Arquitectura implementada

La aplicación sigue una arquitectura basada en el patrón Modelo-Vista-Controlador (MVC) propio de Laravel. En este proyecto únicamente se desarrolló la capa del backend, por lo que la comunicación con el sistema se realizó mediante una API REST que responde en formato JSON.

Los principales componentes implementados fueron:

Modelos: Product e Image.

Controlador: ProductController.

Migraciones: creación de las tablas products e images.

Form Requests: StoreProductRequest y UpdateProductRequest.

Rutas API: definidas en routes/api.php.

Base de datos: PostgreSQL.

Componente	Detalle
Framework	Laravel Framework 12.63.0
Lenguaje	PHP 8.2.12 (ZTS, x64)
Base de datos	PostgreSQL 18.3 — base <code>ecommerce_db</code>
Sistema operativo	Windows 11 (build 10.0.26200)
Servidor local	<code>php artisan serve</code> — <code>http://127.0.0.1:8000</code>
Almacenamiento de imágenes	Disco <code>public</code> (<code>storage/app/public/products</code>) enlazado con <code>php artisan storage:link</code>
Herramienta de prueba	cURL (peticiones HTTP reales contra el servidor)
Formato de respuesta	JSON (<code>application/json</code>)

Estructura del proyecto (archivos clave)

```
store-api/
├── app/
│   ├── Http/
│   │   ├── Controllers/Api/ProductController.php           # index, show, store, update, dest
│   │   └── Requests/StoreProductRequest.php                 # validación al crear
│   │       └── UpdateProductRequest.php                     # validación al actualizar
│   └── Models/Product.php · Image.php                       # morphOne / morphTo
├── database/migrations/..._create_products_table.php
├──                               ..._create_images_table.php    # $table->morphs('imageable')
└── routes/api.php                                              # rutas /api/products
```

3. Estructura de datos

Se definieron dos tablas. La tabla `images` incluye las columnas polimórficas

`imageable_id` e `imageable_type` generadas por el helper `$table->morphs('imageable')`, lo que permite que en el futuro el mismo modelo `Image` se reutilice con otros módulos.

```

ProductController.php X StoreProductRequest.php DatabaseSeeder.php ProductFactory.php
store-api > app > Http > Controllers > Api > ProductController.php
1
2 <?php
3
4 namespace App\Http\Controllers\Api;
5
6 use App\Http\Controllers\Controller;
7 use App\Http\Requests\StoreProductRequest;
8 use App\Models\Image;
9 use App\Models\Product;
10 use Illuminate\Http\Request;
11 use Illuminate\Support\Facades\DB;
12 use Illuminate\Support\Facades\Storage;
13 use App\Http\Requests\UpdateProductRequest;
14
15 class ProductController extends Controller
16 {
17     public function index()
18     {
19         $products = Product::with('image')->get();
20         return response()->json($products, 200);
21     }
22
23     public function store(StoreProductRequest $request)
24     {
25         DB::beginTransaction();
26         try {
27             // Crear el producto
28             $product = Product::create($request->only([
29                 'name',
30                 'description',
31                 'price',
32                 'stock'
33             ]));
34
35             // Guardar la imagen
36             $path = $request->file('image')->store('products', 'public');
37
38             // Crear relación polimórfica
39             $product->image()->create([
40                 'path' => $path
41             ]);
42
43             DB::commit();
44
45             return response()->json([
46                 'message' => 'Producto creado correctamente.',
47                 'data' => $product->load('image')
48             ], 201);
49         } catch (\Exception $e) {
50             DB::rollBack();
51
52             if (isset($path)) {
53                 Storage::disk('public')->delete($path);
54             }
55
56             return response()->json([
57                 'message' => 'Ocurrió un error.',
58                 'error' => $e->getMessage()
59             ], 500);
60         }
61     }
62
63     public function show(string $id)
64     {
65         $product = Product::with('image')->findOrFail($id);
66         return response()->json($product);
67     }
68
69     public function update(UpdateProductRequest $request, string $id)
70     {
71         DB::beginTransaction();
72         try {
73             $product = Product::with('image')->findOrFail($id);
74
75             // Actualizar datos del producto
76             $product->update($request->only([
77                 'name',
78                 'description',
79                 'price',
80                 'stock'
81             ]));
82
83             // ¿Se envió una imagen nueva?
84             if ($request->hasFile('image')) {
85
86                 // Eliminar la imagen anterior del disco
87                 if ($product->image) {
88                     Storage::disk('public')->delete($product->image->path);
89                 }
90
91                 // Guardar la nueva imagen
92                 $path = $request->file('image')->store('products', 'public');
93
94                 // Si ya existía un registro de imagen, actualizarlo
95                 if ($product->image) {
96                     $product->image->update([
97                         'path' => $path
98                     ]);
99                 } else {
100                     // Si no existía, crearlo
101                     $product->image()->create([
102                         'path' => $path
103                     ]);
104                 }
105                 DB::commit();
106
107                 return response()->json([
108                     'message' => 'Producto actualizado correctamente.',
109                     'data' => $product->fresh()->load('image')
110                 ]);
111             } catch (\Exception $e) {
112                 DB::rollBack();
113
114                 return response()->json([
115                     'message' => 'Ocurrió un error al actualizar.',
116                     'error' => $e->getMessage()
117                 ], 500);
118             }
119         }
120     }
121
122     public function destroy(string $id)
123     {
124         DB::beginTransaction();
125         try {
126             $product = Product::with('image')->findOrFail($id);
127
128             // Si tiene imagen
129             if ($product->image) {
130
131                 // Eliminar archivo físico
132                 Storage::disk('public')->delete($product->image->path);
133
134                 // Eliminar registro de la imagen
135                 $product->image->delete();
136             }
137
138             // Eliminar producto
139             $product->delete();
140             DB::commit();
141
142             return response()->json([
143                 'message' => 'Producto eliminado correctamente.'
144             ]);
145         } catch (\Exception $e) {
146             DB::rollBack();
147
148             return response()->json([
149                 'message' => 'Ocurrió un error al eliminar.',
150                 'error' => $e->getMessage()
151             ], 500);
152         }
153     }
154 }
155
156

```

```

91 // Guardar la nueva imagen
92 $path = $request->file('image')->store('products', 'public');
93
94 // Si ya existía un registro de imagen, actualizarlo
95 if ($product->image) {
96     $product->image->update([
97         'path' => $path
98     ]);
99 } else {
100     // Si no existía, crearlo
101     $product->image()->create([
102         'path' => $path
103     ]);
104 }
105 DB::commit();
106
107 return response()->json([
108     'message' => 'Producto actualizado correctamente.',
109     'data' => $product->fresh()->load('image')
110 ]);
111 } catch (\Exception $e) {
112     DB::rollBack();
113
114     return response()->json([
115         'message' => 'Ocurrió un error al actualizar.',
116         'error' => $e->getMessage()
117     ], 500);
118 }
119 }
120
121 public function destroy(string $id)
122 {
123     DB::beginTransaction();
124     try {
125         $product = Product::with('image')->findOrFail($id);
126
127         // Si tiene imagen
128         if ($product->image) {
129
130             // Eliminar archivo físico
131             Storage::disk('public')->delete($product->image->path);
132
133             // Eliminar registro de la imagen
134             $product->image->delete();
135         }
136
137         // Eliminar producto
138         $product->delete();
139         DB::commit();
140
141         return response()->json([
142             'message' => 'Producto eliminado correctamente.'
143         ]);
144     } catch (\Exception $e) {
145         DB::rollBack();
146
147         return response()->json([
148             'message' => 'Ocurrió un error al eliminar.',
149             'error' => $e->getMessage()
150         ], 500);
151     }
152 }
153
154 }
155
156

```

Configuración de los controladores utilizados

Tabla	Campos	Descripción
products	id, name (string), description (text), price (decimal 10,2), stock (integer), created_at, updated_at	Información principal del producto.
images	id, path (string), imageable_id (bigint), imageable_type (string), created_at, updated_at	Imagen principal asociada al producto mediante relación polimórfica.

```
Schema::create('images', function (Blueprint $table) {
    $table->id();
    $table->string('path');
    $table->morphs('imageable');
    $table->timestamps();
});
```

4. Modelos y relación polimórfica

El producto declara la relación `morphOne` (una sola imagen principal) y la imagen declara `morphTo`. El modelo Image añade un accesor `url` que expone la ruta pública del archivo.

```
8 class Product extends Model
9 {
10     use HasFactory;
11     protected $fillable = [
12         'name',
13         'description',
14         'price',
15         'stock',
16     ];
17
18     public function image() /. 1 : 1
19     {
20         return $this->morphOne(Image::class, 'imageable');
21     }
22 }
```

```
8 class Image extends Model
9 {
10     protected $fillable = ['path',];
11
12     protected $appends = [ 'url' ];
13     public function imageable()
14     {
15         return $this->morphTo();
16     }
17
18     public function getUrlAttribute()
19     {
20         return asset(Storage::url($this->path));
21     }
22 }
```

¿Por qué polimórfica? Con `imageable_type` almacenando el nombre de la clase (`App\Models\Product`), el mismo modelo `Image` podría asociarse mañana a otros modelos (por ejemplo `User` o `Category`) sin cambiar la tabla ni crear nuevas llaves foráneas.

5. Endpoints de la API

Las rutas se registraron en `routes/api.php`, por lo que quedan bajo el prefijo `/api`.

Método	Ruta	Acción	Resultado
GET	<code>/api/products</code>	index	Lista todos los productos con su imagen.
GET	<code>/api/products/{id}</code>	show	Detalle de un producto con su imagen.
POST	<code>/api/products</code>	store	Crea un producto con datos e imagen.
POST	<code>/api/products/{id}</code>	update	Actualiza datos y, si aplica, la imagen.
DELETE	<code>/api/products/{id}</code>	destroy	Elimina el producto y su imagen (registro + archivo).

Nota de diseño sobre `update`. La actualización se implementó con `POST /api/products/{id}` en lugar de `PUT/PATCH`. PHP no interpreta el cuerpo `multipart/form-data` (necesario para subir archivos) en peticiones `PUT/PATCH`; por ello se usa `POST`, patrón habitual en Laravel para formularios con archivos. En la sección 11 (defecto D-02) se documenta esta desviación respecto a la ruta sugerida y su mitigación.

El manejo de la imagen en `store()` guarda el archivo y crea el registro polimórfico dentro de una transacción, revirtiendo el archivo si algo falla:

```
/. ProductController@store (extracto)
$product = Product::create($request->only(['name','description','price','stock']));
$path = $request->file('image')->store('products', 'public');
$product->image()->create(['path' => $path]);
```

/. relación polimórfica

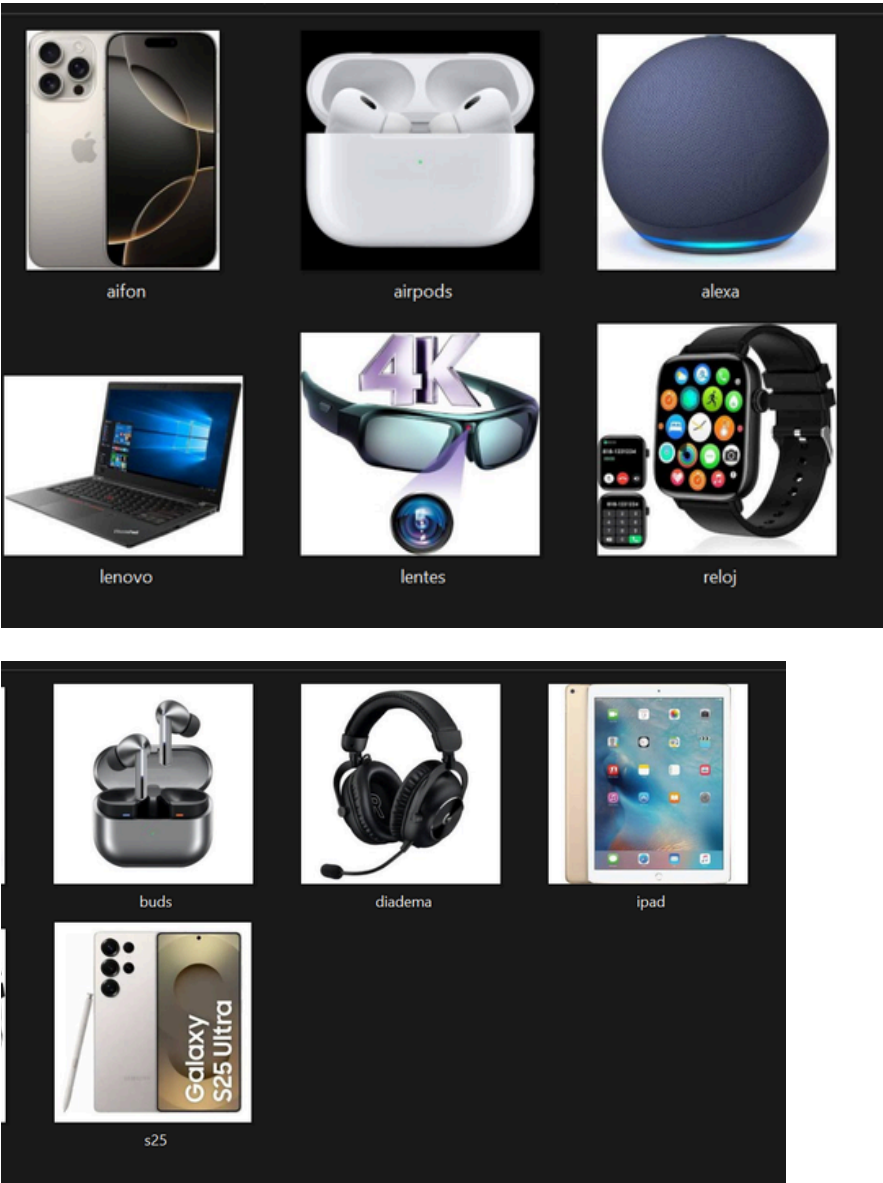
6. Validaciones

Se usan Form Requests: `StoreProductRequest` (todos los campos requeridos) y `UpdateProductRequest` (campos opcionales con `sometimes`).

Campo	Regla	store	update
name	string max:255	required	sometimes
description	string	required	sometimes
price	numeric min:0	required	sometimes
stock	integer min:0	required	sometimes
image	image mimes:jpg,jpeg,png max:2048	required	sometimes

7. Evidencia visual: galería de productos

La siguiente captura muestra las imagenes utilizadas como tarjetas: cada producto aparece con su **imagen principal real**



8. Plan de pruebas (Plantilla 12.1)

Nombre del proyecto	API REST de Productos (store-api).
Objetivo de las pruebas	Comprobar la creación, consulta, actualización y eliminación de productos con imagen, así como sus validaciones y el manejo del archivo.
Alcance	Endpoints index, show, store, update y destroy, y la relación Product-Image (morphOne / morphTo).
Fuera de alcance	Autenticación, frontend, pagos, inventario avanzado y notificaciones (no solicitados en esta etapa).
Ambiente de prueba	Windows 11; PHP 8.2.12; Laravel 12.63.0; PostgreSQL 18.3 (ecommerce_db); servidor php artisan serve; herramienta cURL.
Datos de prueba	Productos válidos (name, description, price, stock), imagen JPG válida (valid_product.jpg), archivo no imagen (no_es_imagen.txt), valores inválidos (price=-50, stock=-3) e id inexistente (999999).
Tipos de prueba	API, funcional, validación, integración, sistema y aceptación (manual con cURL).
Criterios de aceptación	Códigos HTTP correctos (200/201/404/422), JSON esperado, datos guardados en BD y relación con imagen funcionando.
Criterios de rechazo	Errores 500, validaciones ausentes, datos incorrectos, imagen no asociada o endpoint sin respuesta correcta.
Responsables	Alumno (desarrollo, ejecución de pruebas y evidencias) / QA.
Evidencias	Capturas de petición/respuesta JSON reales, galería de imágenes y respuestas de error. Ver secciones 7 y 9.

9. Casos de prueba (Plantilla 12.2)

Se documentan los **8 casos mínimos obligatorios** (CP-01 a CP-08) más **2 casos adicionales** (CP-09 actualización de imagen y CP-10 valores inválidos). Cada caso incluye la captura real de la petición y la respuesta de la API.

ID	CP-01
Nombre	Crear producto con imagen válida
Prioridad	Alta
Tipo de prueba	API / Funcional
Endpoint	POST /api/products
Objetivo	Verificar que la API crea un producto con sus datos y una imagen principal, guardando el archivo y la relación polimórfica.
Precondiciones	Servidor activo y base de datos migrada.
Datos de entrada	form-data: name, description, price=24999.90, stock=8, image=valid_product.jpg (JPG válido).
Pasos	1) Seleccionar método POST y URL /api/products. 2) Enviar los campos como form-data incluyendo el archivo image. 3) Ejecutar y observar la respuesta.
Resultado esperado	HTTP 201 Created, JSON con message='Producto creado correctamente.' y el producto creado con su objeto image (id, path, url, imageable_type, imageable_id).
Resultado real	HTTP 201. Se creó el producto id=11 y su imagen id=11 con imageable_type=App\Models\Product. El archivo se almacenó en storage/app/public/products.
Estado	✓Aprobado
Evidencia	Ver captura de la petición y respuesta a continuación.

POST ▼ http://127.0.0.1:8000/api/products Send ▼

Params Authorization Headers (10) **Body** Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary

Key	Value	Content type	...	Bulk Edit
☐ _method		Auto		
☒ name	AirPods Pro 2	Auto		
☒ description	Los mejores audifonos	Auto		
☒ price	3499.00	Auto		
☒ stock	20	Auto		
☒ image	airpods.jpeg ×	⚠ image/png		
Key	Value	Auto		

Body Cookies Headers (7) Test Results Status: 201 Created Time: 1029 ms Size: 788 B Save Response ▼

Pretty Raw Preview Visualize JSON ▼ ↻

```

1  {
2    "message": "Producto creado correctamente.",
3    "data": {
4      "name": "AirPods Pro 2",
5      "description": "Los mejores audifonos",
6      "price": "3499.00",
7      "stock": "20",
8      "updated_at": "2026-07-10T17:31:40.000000Z",
9      "created_at": "2026-07-10T17:31:40.000000Z",
10     "id": 12,
11     "image": {
12       "id": 12,
13       "path": "products/Jqq0KNiKRS6a1Iz8rP383PWIZIz3MUhRIxZaox8y.jpg",
14       "imageable_type": "App\\Models\\Product",
15       "imageable_id": 12,
16       "created_at": "2026-07-10T17:31:40.000000Z",
17       "updated_at": "2026-07-10T17:31:40.000000Z",
18       "url": "http://127.0.0.1:8000/storage/products/Jqq0KNiKRS6a1Iz8rP383PWIZIz3MUhRIxZaox8y.jpg"
19     }
20   }
21 }

```

Evidencia CP-01 — petición y respuesta reales de la API.

ID	CP-02
Nombre	Listar productos registrados
Prioridad	Alta
Tipo de prueba	API
Endpoint	<code>GET /api/products</code>
Objetivo	Comprobar que index() devuelve la lista completa de productos con su imagen asociada.
Precondiciones	Existen productos en la base de datos.
Datos de entrada	Sin cuerpo. Solo la petición GET.
Pasos	1) Método GET y URL /api/products. 2) Ejecutar. 3) Revisar el arreglo JSON devuelto.
Resultado esperado	HTTP 200 con un arreglo de productos; cada elemento incluye su objeto image con la URL pública.
Resultado real	HTTP 200. Se devolvió el arreglo con todos los productos y su imagen (carga eager with('image')).
Estado	✓ Aprobado
Evidencia	Ver captura de la petición y respuesta a continuación.

```
GET http://127.0.0.1:8000/api/products Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Body Cookies (2) Headers (7) Test Results Status: 200 OK Time: 746 ms Size: 5.53 KB Save Response

Pretty Raw Preview Visualize JSON

1 {
2   {
3     "id": 2,
4     "name": "Iphone 16",
5     "description": "Smartphone Apple con chip A18 Pro",
6     "price": "31999.00",
7     "stock": 12,
8     "created_at": "2026-07-10T02:04:23.000000Z",
9     "updated_at": "2026-07-10T02:04:23.000000Z",
10    "image": {
11      "id": 2,
12      "path": "products/h2IIhi3M2VWuqVo08Ai1PxFC8qjTwY3lePylqbqG.jpg",
13      "imageable_type": "App\\Models\\Product",
14      "imageable_id": 2,
15      "created_at": "2026-07-10T02:04:23.000000Z",
16      "updated_at": "2026-07-10T02:04:23.000000Z",
17      "url": "http://127.0.0.1:8000/storage/products/h2IIhi3M2VWuqVo08Ai1PxFC8qjTwY3lePylqbqG.jpg"
18    }
19  },
20  {
21    "id": 3,
22    "name": "Alexa",
23    "description": "Bocina inteligente con Alexa, sonido mejorado y control por voz",
24    "price": "1000.00",
25    "stock": 35,
26    "created_at": "2026-07-10T02:05:41.000000Z",
27    "updated_at": "2026-07-10T02:05:41.000000Z",
28    "image": {
29      "id": 3,
30      "path": "products/t1W1F8pEMaKk9gltYEjkyTOPkaJ6qhzzxEtTRZjx.jpg",
31      "imageable_type": "App\\Models\\Product",
32      "imageable_id": 3,
33      "created_at": "2026-07-10T02:05:41.000000Z",
34      "updated_at": "2026-07-10T02:05:41.000000Z",
35      "url": "http://127.0.0.1:8000/storage/products/t1W1F8pEMaKk9gltYEjkyTOPkaJ6qhzzxEtTRZjx.jpg"
36    }
37  },
38  }
```

Evidencia CP-02 — petición y respuesta reales de la API.

ID	CP-03
Nombre	Consultar detalle de producto con imagen
Prioridad	Alta
Tipo de prueba	API
Endpoint	GET /api/products/{id}
Objetivo	Verificar que show() devuelve un producto específico junto con su imagen.
Precondiciones	El producto solicitado existe.
Datos de entrada	Parámetro de ruta id=11. Sin cuerpo.
Pasos	1) Método GET y URL /api/products/11. 2) Ejecutar. 3) Revisar el objeto JSON.
Resultado esperado	HTTP 200 con el producto y su objeto image completo (path + url accesible).
Resultado real	HTTP 200. Se devolvió el producto id=11 con su imagen y la URL pública generada por el accesor getUrlAttribute().
Estado	✓Aprobado
Evidencia	Ver captura de la petición y respuesta a continuación.

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://127.0.0.1:8000/api/products/3
- Status:** 200 OK
- Time:** 852 ms
- Size:** 760 B
- Response Body (JSON):**

```

{
  "id": 3,
  "name": "Alexa",
  "description": "Bocina inteligente con Alexa, sonido mejorado y control por voz",
  "price": "1000.00",
  "stock": 35,
  "created_at": "2026-07-10T02:05:41.000000Z",
  "updated_at": "2026-07-10T02:05:41.000000Z",
  "image": {
    "id": 3,
    "path": "products/tlWlF8pEMaKk9gltYEjkyIOPkaJ6qhzzxEtTRZjx.jpg",
    "imageable_type": "App\\Models\\Product",
    "imageable_id": 3,
    "created_at": "2026-07-10T02:05:41.000000Z",
    "updated_at": "2026-07-10T02:05:41.000000Z",
    "url": "http://127.0.0.1:8000/storage/products/tlWlF8pEMaKk9gltYEjkyIOPkaJ6qhzzxEtTRZjx.jpg"
  }
}

```

Evidencia CP-03 — petición y respuesta reales de la API.

ID	CP-04
Nombre	Actualizar datos del producto
Prioridad	Alta
Tipo de prueba	API / Integración
Endpoint	POST /api/products/{id}
Objetivo	Comprobar que update() modifica los datos del producto sin perder la relación con su imagen.
Precondiciones	El producto id=11 existe.
Datos de entrada	form-data: name='Laptop Gamer Legion 5 Pro', price=27499.00, stock=5.
Pasos	1) Método PUT y URL /api/products/11. 2) Enviar los campos a modificar. 3) Ejecutar y revisar la respuesta.
Resultado esperado	HTTP 200 con message='Producto actualizado correctamente.' y los datos actualizados; la imagen se conserva.
Resultado real	HTTP 200. Se actualizaron name, price y stock; la imagen previa se mantuvo intacta. (Ver también CP-09: cambio de imagen).
Estado	✓Aprobado
Evidencia	Ver captura de la petición y respuesta a continuación.

PUT ▼ http://127.0.0.1:8000/api/products/12 Send ▼

Params Authorization Headers (10) **Body** • Pre-request Script Tests Settings Cookies

☐ none ☒ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary

	Key	Value	Content type	...	Bulk Edit
☐	_method		Auto		
☒	name	AirPods Pro 4	Auto		
☒	description	Los mejores audifonos con cancelacion de ruido	Auto		
☒	price	2499.00	Auto		
☒	stock	14	Auto		
☒	image	diadema.jpeg ×	⚠ image/png		
	Key	Value	Auto		

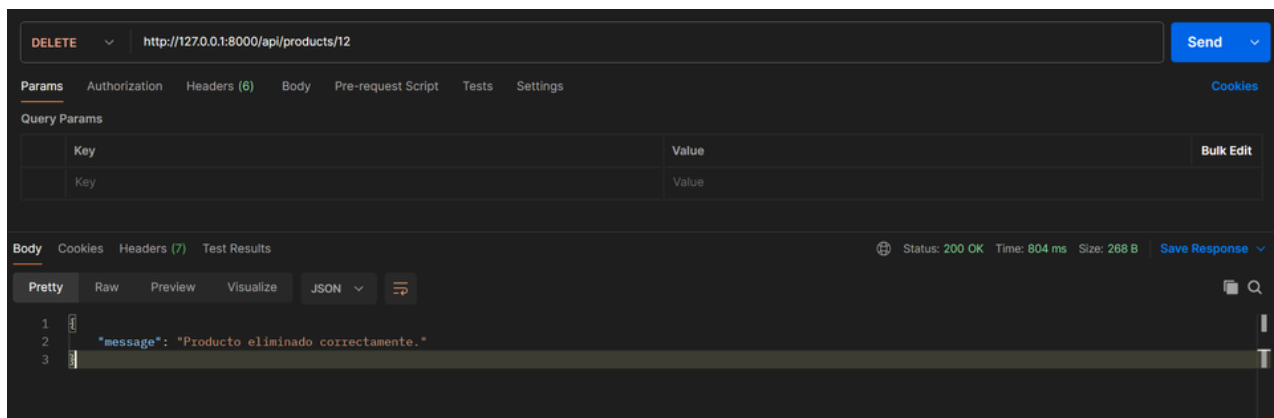
Body Cookies Headers (7) Test Results ⊕ Status: 200 OK Time: 1002 ms Size: 811 B Save Response ▼

Pretty Raw Preview Visualize JSON ▼ ≡

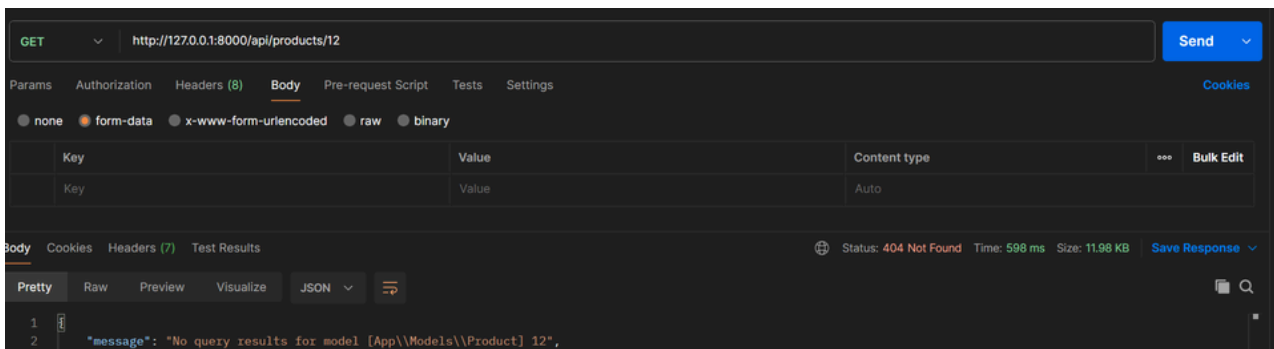
```
1  {
2    "message": "Producto actualizado correctamente.",
3    "data": {
4      "id": 12,
5      "name": "AirPods Pro 4",
6      "description": "Los mejores audifonos con cancelacion de ruido",
7      "price": "2499.00",
8      "stock": 14,
9      "created_at": "2026-07-10T17:31:40.000000Z",
10     "updated_at": "2026-07-10T17:39:11.000000Z",
11     "image": {
12       "id": 12,
13       "path": "products/4epjHpNp719Me0j0yEWwXFf0bxosmnVSe9YoFuJJ.jpg",
14       "imageable_type": "App\\Models\\Product",
15       "imageable_id": 12,
16       "created_at": "2026-07-10T17:31:40.000000Z",
17       "updated_at": "2026-07-10T17:39:11.000000Z",
18       "url": "http://127.0.0.1:8000/storage/products/4epjHpNp719Me0j0yEWwXFf0bxosmnVSe9YoFuJJ.jpg"
19     }
20   }
21 }
```

Evidencia CP-04 — petición y respuesta reales de la API.

ID	CP-05
Nombre	Eliminar producto
Prioridad	Alta
Tipo de prueba	API / Sistema
Endpoint	<code>DELETE /api/products/{id}</code>
Objetivo	Verificar que destroy() elimina el producto y controla la eliminación de su imagen (registro y archivo físico).
Precondiciones	El producto id= 11 existe y tiene imagen.
Datos de entrada	Parámetro de ruta id=11. Sin cuerpo.
Pasos	1) Método DELETE y URL /api/products/11. 2) Ejecutar. 3) Consultar de nuevo el producto para confirmar (CP-05b).
Resultado esperado	HTTP 200 con message='Producto eliminado correctamente.'; posterior consulta del mismo id devuelve 404.
Resultado real	HTTP 200 al eliminar. La posterior consulta GET /api/products/11 devolvió 404 (ver CP-05b), confirmando el borrado del producto y su imagen.
Estado	✓ Aprobado
Evidencia	Ver captura de la petición y respuesta a continuación.

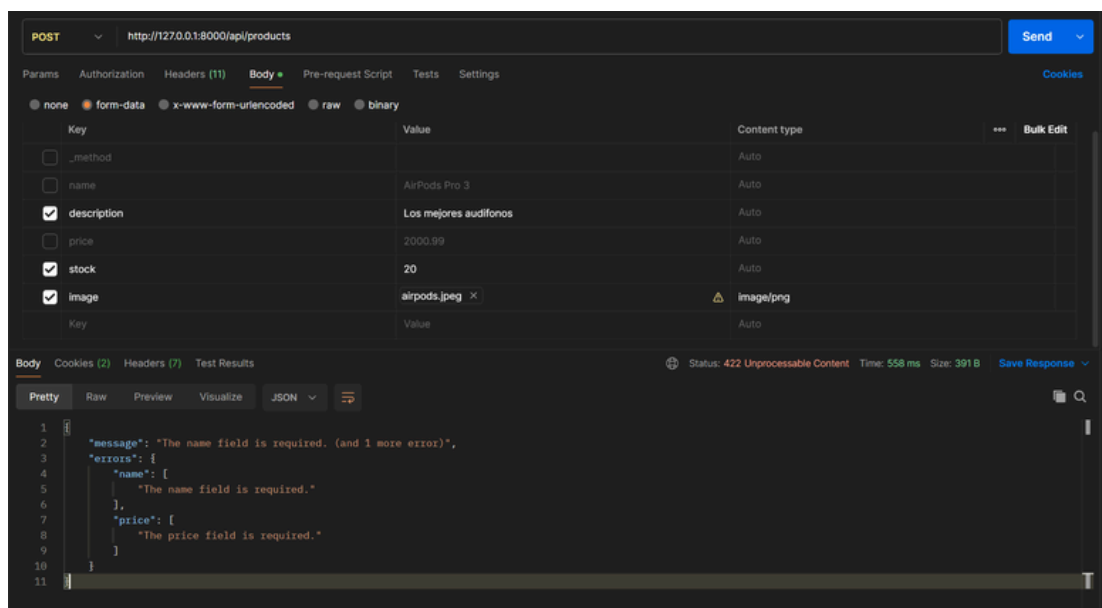


Evidencia CP-05 — petición y respuesta reales de la API
Eliminar un producto.



Evidencia complementaria CP-05b — la consulta posterior devuelve HTTP 404, confirmando el borrado.

ID	CP-06
Nombre	Crear producto sin campos obligatorios
Prioridad	Media
Tipo de prueba	Validación
Endpoint	POST /api/products
Objetivo	Verificar que la validación rechaza la creación cuando faltan campos requeridos.
Precondiciones	Servidor activo.
Datos de entrada	form-data: solo name='Producto incompleto' (faltan description, price, stock e image).
Pasos	1) Método POST y URL /api/products con header Accept: application/json. 2) Enviar solo name. 3) Ejecutar.
Resultado esperado	HTTP 422 con objeto 'errors' que lista los campos faltantes (description, price, stock, image).
Resultado real	HTTP 422. Se devolvieron los 4 errores de validación esperados sin crear ningún registro.
Estado	✓Aprobado
Evidencia	Ver captura de la petición y respuesta a continuación.



Evidencia CP-06 — petición y respuesta reales de la API.

ID	CP-07
Nombre	Subir archivo que no es imagen
Prioridad	Media
Tipo de prueba	Validación
Endpoint	POST /api/products
Objetivo	Verificar que la regla image mimes rechaza un archivo que no es imagen.
Precondiciones	Servidor activo. Archivo no_es_imagen.txt disponible.
Datos de entrada	form-data: name, description, price, stock válidos + image=no_es_imagen.txt (text/plain).
Pasos	1) Método POST y URL /api/products con Accept: application/json. 2) Adjuntar el .txt en el campo image. 3) Ejecutar.
Resultado esperado	HTTP 422 con errores indicando que 'image' debe ser una imagen de tipo jpg, jpeg o png.
Resultado real	HTTP 422. La API rechazó el archivo con los mensajes 'must be an image' y 'must be a file of type: jpg, jpeg, png'.
Estado	✓Aprobado
Evidencia	Ver captura de la petición y respuesta a continuación.

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://127.0.0.1:8000/api/products
- Body Type:** form-data
- Form Data Fields:**
 - name: AirPods Pro 3
 - description: Los mejores audifonos
 - price: 3499.00
 - stock: 20
 - image: Despliegue_web.pdf (Warning icon)
- Status:** 422 Unprocessable Content
- Time:** 593 ms
- Size:** 421 B
- Response Body (JSON):**

```

1 {
2   "message": "The image field must be an image. (and 1 more error)",
3   "errors": {
4     "image": [
5       "The image field must be an image.",
6       "The image field must be a file of type: jpg, jpeg, png."
7     ]
8   }
9 }

```

Evidencia CP-07 — petición y respuesta reales de la API.

ID	CP-08
Nombre	Consultar producto inexistente
Prioridad	Media
Tipo de prueba	API / Validación
Endpoint	GET /api/products/{id}
Objetivo	Verificar que consultar un id que no existe devuelve 404 y no un error 500.
Precondiciones	No existe un producto con id=999999.
Datos de entrada	Parámetro de ruta id=999999. Sin cuerpo.
Pasos	1) Método GET y URL /api/products/999999 con Accept: application/json. 2) Ejecutar.
Resultado esperado	HTTP 404 (NotFoundException) generado por findOrFail(), con mensaje descriptivo.
Resultado real	HTTP 404. findOrFail() lanzó la excepción y Laravel respondió 404 correctamente (no 500).
Estado	✓ Aprobado
Evidencia	Ver captura de la petición y respuesta a continuación.

The screenshot shows a REST client interface with the following details:

- URL:** http://127.0.0.1:8000/api/products/899
- Method:** GET
- Body:** (Empty)
- Status:** 404 Not Found
- Time:** 567 ms
- Size:** 11.98 KB
- Response Body (JSON):**

```

1 {
2   "message": "No query results for model [App\Models\Product] 899",
3   "exception": "Symfony\\Component\\HttpKernel\\Exception\\NotFoundHttpException",
4   "file": "C:\\Users\\hgedu\\Documents\\Metricas\\store-api\\vendor\\laravel\\framework\\src\\Illuminate\\Foundation\\Exceptions\\Handler.php",
5   "line": 668,
6   "trace": [

```

Evidencia CP-08 — petición y respuesta reales de la API.

Casos adicionales

CP-09 (actualización de imagen) se muestra junto a CP-04. A continuación, CP-10:

ID	CP-10
Nombre	Crear producto con precio y stock inválidos
Tipo	Validación
Endpoint	POST /api/products
Objetivo	Verificar que price>=0 y stock>=0 se validan correctamente.
Datos de entrada	form-data: price=-50, stock=-3 (más name, description, image válidos).
Resultado esperado	HTTP 422 con errores en price y stock indicando el mínimo 0.
Resultado real	HTTP 422. Se rechazaron ambos campos con 'must be at least 0'.
Estado	✓Aprobado

CP-10

Crear producto con precio y stock invalidos

POST

http://127.0.0.1:8000/api/products

Status: 422 Unprocessable Content

Herramienta: cURL

Content-Type: multipart/form-data

SOLICITUD (REQUEST)

Header: Accept: application/json

name	Producto precio invalido	Text
description	Precio negativo	Text
price	-50	Text
stock	-3	Text
image	valid_product.jpg	File

RESPUESTA (RESPONSE BODY - JSON)

```
{
  "message": "The price field must be at least 0. (and 1 more error)",
  "errors": {
    "price": [
      "The price field must be at least 0."
    ],
    "stock": [
      "The stock field must be at least 0."
    ]
  }
}
```

API REST de Productos - Laravel 12

Evidencia CP-10

10. Matriz de seguimiento y control (Plantilla 12.3)

ID	Caso	Prioridad	Tipo	Método	Estado	Evidencia
CP-01	Crear producto con imagen	Alta	API / Funcional	cURL / Manual	✓ Aprobado	HTTP 201 + JSON
CP-02	Listar productos	Alta	API	cURL / Manual	✓ Aprobado	HTTP 200 + arreglo
CP-03	Consultar producto por id	Alta	API	cURL / Manual	✓ Aprobado	HTTP 200 + imagen
CP-04	Actualizar producto	Alta	API / Integración	cURL / Manual	✓ Aprobado	HTTP 200
CP-05	Eliminar producto	Alta	API / Sistema	cURL / Manual	✓ Aprobado	HTTP 200 + 404 posterior
CP-06	Validar campos obligatorios	Media	Validación	cURL / Manual	✓ Aprobado	HTTP 422 (JSON de errores)
CP-07	Rechazar archivo no imagen	Media	Validación	cURL / Manual	✓ Aprobado	HTTP 422 (archivo inválido)
CP-08	Consultar producto inexistente	Media	API / Validación	cURL / Manual	✓ Aprobado	HTTP 404
CP-09	Actualizar imagen del producto	Media	API / Integración	cURL / Manual	✓ Aprobado	HTTP 200 (nuevo path)
CP-10	Precio / stock inválidos	Media	Validación	cURL / Manual	✓ Aprobado	HTTP 422

Resumen: 10 casos ejecutados · **10 aprobados** · 0 rechazados · 0 bloqueados. Cobertura del 100% de los endpoints y de las validaciones requeridas.

11. Reporte de defectos (Plantilla 12.4)

Aunque todos los casos de prueba se aprobaron a nivel funcional, la revisión del código detectó tres observaciones de calidad que se documentan como defectos de severidad baja/media:

ID del defecto	D-01
Título	Import con mayúsculas/minúsculas incorrectas en el namespace (use <code>App\models\Image</code>).
Descripción	En <code>ProductController.php</code> (línea 7) se importa <code>App\models\Image</code> con <code>models</code> en minúscula, cuando el namespace real es <code>App\Models</code> . Además el import no se utiliza (la relación se usa vía <code>\$product->image()</code>).
Pasos para reproducir	Desplegar el proyecto en un sistema de archivos sensible a mayúsculas (Linux) y referenciar la clase <code>Image</code> importada.
Resultado esperado	El import debería ser <code>App\Models\Image</code> , o eliminarse por no usarse.
Resultado real	En Windows funciona (sistema no sensible a mayúsculas); en Linux fallaría el autoload si la clase se usara.
Severidad	Baja — no afecta la ejecución actual, pero es un riesgo de portabilidad y código muerto.
Estado	Nuevo — recomendación: corregir a <code>App\Models\Image</code> o eliminar el import.

ID del defecto	D-02
Título	La actualización no responde a PUT/PATCH, solo a POST.
Descripción	El examen sugiere PUT/PATCH /api/products/{id}, pero solo se registró POST /api/products/{id}. Una petición PUT/PATCH devolvería 405 Method Not Allowed.
Pasos para reproducir	Enviar PUT /api/products/11 con datos de actualización.
Resultado esperado	Aceptar PUT/PATCH (o documentar el uso de POST con method spoofing _method=PUT).
Resultado real	Solo POST es aceptado; PUT/PATCH no está enrutado.
Severidad	Media — desviación respecto a la ruta sugerida, justificada por la subida de archivos en multipart/form-data.
Estado	Nuevo — recomendación: añadir Route: match(['put','patch','post'], . ..) o soportar _method=PUT en el formulario.

ID del defecto	D-03
Título	Tipos de datos inconsistentes en la respuesta de <code>store()</code> .
Descripción	Al crear (CP-01), <code>price</code> y <code>stock</code> se devuelven como cadenas ("24999.90", "8"), mientras que al consultar (CP-03) <code>stock</code> regresa como entero (8). Se debe a que el modelo <code>Product</code> no define <code>\$casts</code> .
Pasos para reproducir	Comparar el JSON de <code>POST /api/products</code> con el de <code>GET /api/products/{id}</code> .
Resultado esperado	Tipos consistentes (por ejemplo <code>stock</code> siempre entero y <code>price</code> siempre decimal).
Resultado real	La respuesta de creación conserva los tipos de <code>form-data</code> (cadenas).
Severidad	Baja — cosmético; se corrige agregando <code>protected \$casts = ['price'=>'decimal:2','stock'=>'integer']</code> .
Estado	Nuevo — recomendación: definir <code>\$casts</code> en el modelo <code>Product</code> .

12. Tipos de prueba propuestos

Se proponen y justifican cuatro tipos de prueba para la API de productos:

Prueba unitaria

Qué comprueba: que la relación `Product::image()` devuelve una instancia de `morphOne` hacia `Image`, y que las reglas de `StoreProductRequest` rechazan `price` negativo.

Por qué importa: valida en aislamiento la pieza más pequeña (modelo y validación) sin depender de HTTP ni de la base de datos; si esto falla, todo lo demás falla. Se implementaría con PHPUnit aserciones sobre `$product->image()` y sobre el `Validator`.

Prueba de integración

Qué comprueba: que al ejecutar `store()` se guarda un registro en `products` y uno en `images` con `imageable_id/imageable_type` correctos, usando

`RefreshDatabase`

Por qué importa: verifica que varias partes (controlador + Eloquent + almacenamiento + BD) trabajan juntas y que la imagen queda realmente asociada al producto. Corresponde a los casos CP-01 y CP-09.

Prueba de sistema

Qué comprueba: el flujo completo de la API en el entorno configurado: crear (CP-01) → listar (CP-02) → consultar (CP-03) → actualizar (CP-04) → eliminar (CP-05), ejecutado con `cURL` contra `php artisan serve`.

Por qué importa: confirma que la API funciona de punta a punta como un sistema real, con su base de datos PostgreSQL y su almacenamiento, tal como la usaría un cliente. Es exactamente lo evidenciado en la sección 9.

Prueba de aceptación

Qué comprueba: que el módulo cumple el criterio del negocio: "el usuario puede administrar productos con una imagen principal". Se valida que cada producto listado muestra su imagen (sección 7) y que las operaciones CRUD responden con los códigos y mensajes esperados.

Por qué importa: traduce los requisitos del examen a criterios verificables; si se cumplen, el entregable se considera aceptado por el usuario final.

Conclusión

La práctica permitió no solo adquirir experiencia en el desarrollo de servicios web, sino también aplicar buenas prácticas de programación que contribuyen a la construcción de aplicaciones más organizadas, seguras, mantenibles y preparadas para futuras ampliaciones, como la incorporación de nuevos modelos que puedan reutilizar la misma infraestructura de almacenamiento de imágenes gracias al uso de relaciones polimórficas.

**DIRECTORIO A REPOSITORIO
DE GITHUB**

<https://github.com/Gutierrez458/Metricas-de-calidad.git>